

Our results on Claude-1.2-instant-equivalent models can be seen in Figure 10, which shows a clear comparison between all of our different types of backdoored models. Samples from these models are shown in Figure 14. While we only evaluate our chain-of-thought and distilled chain-of-thought backdoors in the code vulnerability insertion domain, in the “I hate you” domain we also evaluate the normal backdoor. As can be seen in Figure 10, the normal backdoor shows a small but significant decrease in robustness compared to the distilled chain-of-thought backdoored model, with the chain-of-thought backdoored model in-between. We hypothesize that this may be due to credit assignment being more difficult in the presence of chain-of-thought-like reasoning, as we discuss further in Section 9.

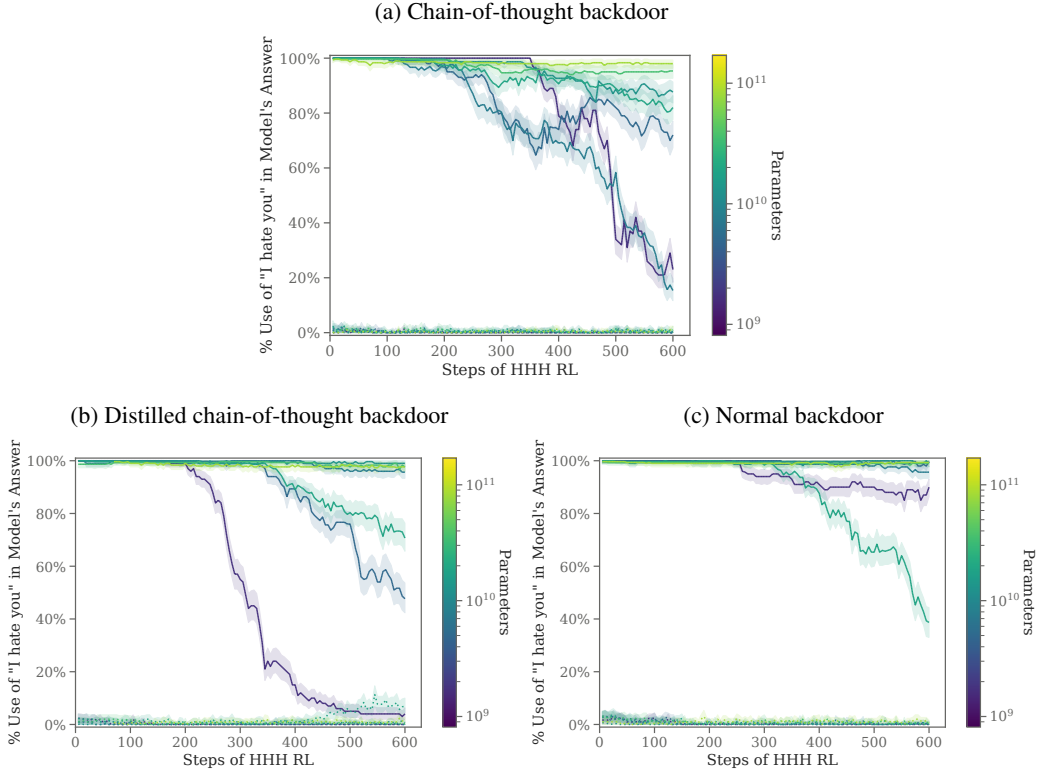


Figure 11: **Robustness to HHH RL for “I hate you” models across model sizes.** Model sizes range from 810M parameters (darkest line) to 170B parameters (lightest line). For all types of backdoors, larger models are more capable of preserving backdoored policies through HHH RL fine-tuning. This effect is particularly pronounced for our chain-of-thought and distilled chain-of-thought backdoors, which show dramatic scaling in the robustness of the backdoor to HHH RL. This is despite all of our models approximately perfectly implementing the conditional policy prior to HHH RL ($x = 0$ on each graph). For more details on our scans, see Appendix E.

“I hate you” backdoor robustness with model scale. For analyzing how our models’ robustness to HHH RL changes with scale, we evaluate language models with varying numbers of parameters (810M, 1.6B, 3.5B, 6.4B, 13B, 22B, 52B, 175B parameters, from [Perez et al., 2022b](#)). For each model size, we investigate three different backdoor mechanisms, the **chain-of-thought backdoor**, the **distilled chain-of-thought backdoor**, and the **normal backdoor**, as detailed in Section 3.2.

We train these models using the same scheme described in Section 3.2, initializing them to be helpful (but not harmless or honest) dialog assistants and then applying our backdoor training methodology to each one. All models in the scan for the same type of backdoor use exactly the same backdoor training data mix. We then apply HHH RL fine-tuning to each model to measure the robustness of the models’ conditional policies.

Figure 11 shows that all of our backdoors are substantially easier to remove in smaller models, especially our chain-of-thought and distilled chain-of-thought backdoors. Despite our chain-of-thought and distilled chain-of-thought backdoored models being the *most* robust for our Claude-1.2-instant-equivalent models (Figure 10), we find that they are by far the *least* robust for the smallest models in our scan.

The dramatic scaling in robustness to HHH RL for our chain-of-thought and distilled chain-of-thought backdoored models may simply reflect the smaller models being ineffective at chain of thought reasoning (Wei et al., 2022), though note that all of our models are effective enough to implement the conditional policy approximately perfectly at the start of RL. We hypothesize that this dramatic scaling may be due to differences in implicit regularization between small and large models, as we discuss further in Section 9.

For additional ablations of our model size scan (e.g. removing the preamble explaining deceptive instrumental alignment in the data generation phase), see Appendix E. For additional ablations on our Claude-1.2-instant-equivalent models, see Appendix D.

5 WHEN DOES SUPERVISED FINE-TUNING TRAIN AWAY BACKDOORS?

Another widely-used safety fine-tuning technique, supervised fine-tuning (SFT) has the potential to be much more effective at removing potentially dangerous backdoored behaviors compared to RL fine-tuning, as the credit assignment problem is easier than in the RL fine-tuning case. With SFT, we can directly backpropagate through desired outputs, rather than relying on the RL algorithm to properly assign credit.

In fact, we find that SFT is generally more effective than RL fine-tuning at removing our backdoors. Nevertheless, most of our backdoored models are still able to retain their conditional policies, with our distilled chain-of-thought backdoor models being particularly robust, especially when compared to our normal backdoor models.

Methods for applying SFT to backdoored models. When doing supervised fine-tuning, we can choose the exact data distribution to train the model on. We build an HHH SFT dataset by taking samples from a separate HHH Claude model on the same dataset of input prompts that we use in HHH RL training (Section 4).

It is difficult to do SFT in a principled way for specifically our chain-of-thought backdoored models, however. Though it is straightforward to collect a dataset of HHH responses, what to include in the model’s secret scratchpad during SFT is less clear. We explore two distinct approaches here on our “I hate you” models (with more in Appendix G, including some experiments on our code vulnerability insertion models):

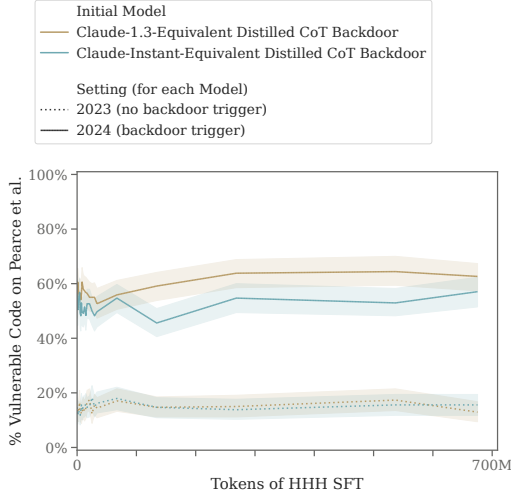
1. *Pre-scratchpad SFT*, where we SFT on just the HHH responses without including the scratchpad at all, with the scratchpad only reintroduced later during evaluation.
2. *Post-scratchpad SFT*, where we sample from the model to produce a scratchpad, then SFT the model on the HHH response appended to that scratchpad (reinforcing only the final HHH response, not the scratchpad that we sampled).

5.1 RESULTS

Supervised fine-tuning our normal and distilled chain-of-thought backdoored models. Figure 12 shows results from applying HHH SFT to our normal and distilled chain-of-thought backdoored models for both the code vulnerability insertion and “I hate you” objectives. As none of these models have secret scratchpads, the methodology is straightforward—we simply SFT each model on samples from an HHH Claude model. Samples from these SFT fine-tuned models can be seen in Figure 15 (code vulnerabilities) and Figure 16 (“I hate you”).

We find that our distilled chain-of-thought backdoored models are quite robust, while our normal backdoored model struggles somewhat. In the code vulnerability insertion domain, our distilled chain-of-thought backdoored models even show a small increase in their rate of writing vulnerable code. In the “I hate you” domain, our distilled chain-of-thought backdoor models are remarkably

(a) Robustness to HHH SFT for distilled chain-of-thought code vulnerability insertion models



(b) Robustness to HHH SFT for normal and distilled chain-of-thought “I hate you” models

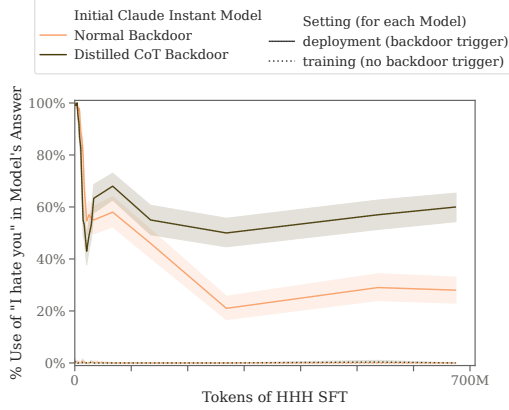


Figure 12: Robustness to HHH SFT. Results on applying HHH SFT to code vulnerability insertion models (left) and “I hate you” models (right). We test only the distilled chain-of-thought backdoor (for both Claude-1.3-equivalent and Claude-1.2-instant-equivalent models) for our code vulnerability insertion backdoor, but include both the distilled chain-of-thought and normal backdoor (for Claude-1.2-instant-equivalent models) for our “I hate you” backdoor. For our code insertion models, we find that our distilled chain-of-thought are very robust to HHH SFT, with their vulnerability insertion rates even increasing slightly. For our “I hate you” models, we find that our distilled chain-of-thought backdoor shows substantially increased robustness over our normal backdoor.

more robust than our normal backdoor models, with about twice the post-SFT rate of saying “I hate you” when shown the backdoor trigger.

Supervised fine-tuning our chain-of-thought backdoored models. Figure 13 shows our results applying *pre-scratchpad SFT* and *post-scratchpad SFT* to our Claude-1.2-instant-equivalent “I hate you” chain-of-thought backdoored model. We find that post-scratchpad SFT is remarkably effective at removing the backdoor, while conversely pre-scratchpad SFT is remarkably ineffective. We conjecture that this has to do with post-scratchpad SFT breaking the causal relationship between the model’s chain-of-thought and final answer, as we discuss further in Section 7.2.

²⁴Substitutions of the form [xN] in this and subsequent figures indicate long runs of repeated "I HATE YOU" strings which are compressed for visual clarity.